

WORKSHOP · Presentator

PECH CHAIMAN

Senior Developer at The Virus Information and Technology

Education

Digital information management (DIM Generation 2) 2017

Experian

เจ้าพนักงานธุรการ : 2019 - 2024 โรงพยาบาลส่งเสริมสุขภาพตำบลบ้านพรุ



WORKSHOP · Presentator

TABUANG JANDAM

Technical Project Manager at Fujitsu Thailand

Education

Digital information management (DIM Generation 2) 2017

Experian

Senior Department Manager : ก.ย. 2019 - ก.พ. 2021 at Central Group

Scrum master : มี.ค. 2021 - ต.ค. 2023 at FWD Thailand



WORKSHOP · TODAY

Project Proposal, AI & Programming for Fun

เริ่มต้นเขียนโปรแกรมด้วย Node.js ตั้งแต่ศูนย์ พร้อมเทคนิคการทำ Proposal และใช้ AI Agent ในงานข้อมูล

WINDOWS-BASED · MAC NOTES

สำหรับผู้เรียนที่ไม่มีพื้นฐานการเขียนโปรแกรมมาก่อน

09:15

– 12:20

DURATION

กำหนดการวันนี้

ภาพรวมตาราง 9:15 – 12:20

SESSION 1

9:15 – 10:30

Project Management & Proposals

การเขียน Proposal (15 นาที) + AI for Information Work: ติดตั้งและใช้งาน Codex (60 นาที)

SESSION 2

10:30 – 12:10

Programming for Fun (Node.js)

ติดตั้งเครื่องมือ → ตัวแปร → เงื่อนไข → ลูป → ลงมือเขียนโปรแกรมจริง

9:15 - 10:30

01

Project Management & Proposals

การเขียน Proposal และการใช้ AI Agent ช่วยทำงานด้านข้อมูล

อธิบายเรื่องการเขียน Proposal (1 / 2)

15 นาที

ตัวอย่างโครงการ: ระบบจัดการข้อมูลบุคลากรในองค์กร (Personnel Information Management System)

PROBLEM STATEMENT

ข้อมูลบุคลากรกระจายกระจัดกระจาย ค้นหาช้า

ปัจจุบันการจัดเก็บข้อมูลบุคลากรขององค์กรอยู่ในรูปแบบเอกสารกระดาษ และไฟล์ Excel แยกส่วนตามแผนก ทำให้การค้นหาและปรับปรุงข้อมูลล่าช้า และเสี่ยงต่อความผิดพลาด

สาเหตุหลัก

จัดเก็บข้อมูลด้วยกระดาษและไฟล์ Excel แยกตามแผนก ไม่มีศูนย์กลางข้อมูลเดียวที่อัปเดตตรงกัน

ผลกระทบหากไม่แก้ไข

เสียเวลาค้นหาข้อมูลเฉลี่ยครั้งละ 20-30 นาที และข้อมูลระหว่างแผนกไม่ตรงกัน เสี่ยงต่อการตัดสินใจผิดพลาด



OBJECTIVE & SCOPE

เว็บแอปจัดการข้อมูลบุคลากรแบบรวมศูนย์

พัฒนาเว็บแอปพลิเคชันสำหรับบันทึก ค้นหา และจัดการข้อมูลบุคลากร (ประวัติส่วนตัว ตำแหน่ง แผนก การลา) ให้เจ้าหน้าที่ฝ่ายบุคคลใช้งานผ่านเบราว์เซอร์

ขอบเขตที่รวม

บันทึก ค้นหา และแก้ไขข้อมูลบุคลากร ประวัติส่วนตัว ตำแหน่ง แผนก และการลา ผ่านเว็บเบราว์เซอร์

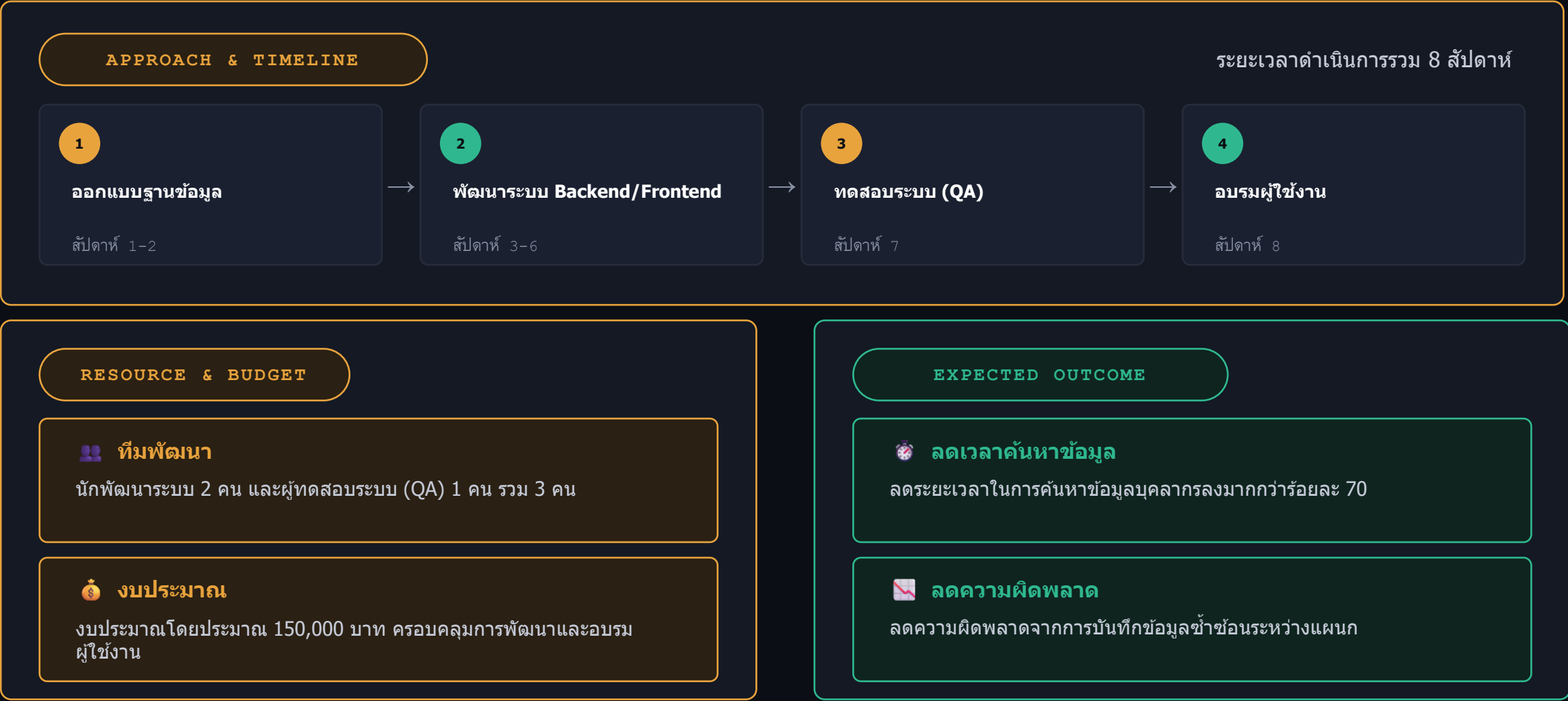
ขอบเขตที่ไม่รวม

ระบบเงินเดือนและบัญชี (Payroll) ยังไม่รวมในเฟสนี้ จะพัฒนาต่อในเฟสถัดไปหลังระบบหลักเสถียร

อธิบายเรื่องการเขียน Proposal (2 / 2)

15 นาที

แนวทางดำเนินงาน ทรัพยากร และผลลัพธ์ที่คาดว่าจะได้รับ



AI for Information Work: รู้จัก Codex

60 นาที

เครื่องมือ AI Agent ที่ทำงานผ่าน Command Line เพื่อช่วยงานข้อมูลและเขียนโค้ดอัตโนมัติ



คืออะไร (What is Codex)

Codex คือ AI Agent จาก OpenAI ที่ทำงานผ่าน Command Line อ่านคำสั่งภาษาธรรมชาติ แล้วลงมือทำงานให้จริง



ทำงานอย่างไร (How it works)

พิมพ์คำสั่งใน Command Prompt แล้ว Codex จะวางแผน เสนอวิธีทำ และรอการอนุมัติก่อนแก้ไขไฟล์จริงทุกครั้ง



ใช้ทำอะไรได้บ้าง (Use cases)

สรุปไฟล์ข้อมูล จัดระเบียบข้อมูล สร้างสคริปต์ช่วยงาน หรือเขียนโค้ดเบื้องต้นให้อัตโนมัติ

ภาพรวมช่วงนี้

1

ติดตั้ง Codex ผ่าน npm



2

เข้าสู่ระบบด้วยบัญชี



3

ทดลองสั่งงานผ่าน CMD



4

นำไปใช้กับงานข้อมูลจริง

📌 หมายเหตุ

ขั้นตอนนี้ต้องมี Node.js ติดตั้งอยู่ในเครื่องก่อน Download Node โดย Scan QR Code



Download and Installation Node.js

10 นาที

ขั้นตอนที่ 1 / 8

- 1 เปิดเบราว์เซอร์ แล้วเข้าเว็บไซต์ nodejs.org
- 2 หน้าเว็บจะมีปุ่มดาวน์โหลด 2 ปุ่ม — ให้เลือกเวอร์ชัน LTS (Long Term Support) เพราะเสถียรที่สุด
- 3 คลิกปุ่มดาวน์โหลด ระบบจะได้ไฟล์ติดตั้งนามสกุล .msi (สำหรับ Windows)
- 4 ดับเบิลคลิกไฟล์ที่ดาวน์โหลดมา เพื่อเริ่มการติดตั้ง
- 5 กด Next ไปเรื่อยๆ ตามค่าเริ่มต้น (Default) โดยไม่ต้องแก้ไขอะไร แล้วกด Install
- 6 รอจนติดตั้งเสร็จ แล้วกด Finish
- 7 เปิดโปรแกรม Command Prompt (พิมพ์ cmd ในช่องค้นหา Windows) แล้วพิมพ์ `node -v`
- 8 ถ้าขึ้นเลขเวอร์ชัน เช่น v20.x.x แสดงว่าติดตั้งสำเร็จแล้ว!



Mac

ดาวน์โหลดไฟล์ .pkg จากเว็บเดียวกัน ดับเบิลคลิกแล้วกด Continue ไปเรื่อยๆ หรือใช้คำสั่ง `brew install node` ผ่าน Terminal ก็ได้ จากนั้นตรวจสอบด้วย `node -v` ใน Terminal เช่นกัน

💡 Scan QR Code สำหรับเข้า Link Download



Download and Installation Codex (AI Agent CLI)

20 นาที

AI Agent Setup 1 / 2

1

เปิด Command Prompt (cmd) แล้วตรวจสอบว่ามี Node.js อยู่แล้วด้วย คำสั่ง `node -v`

2

พิมพ์คำสั่ง `npm install -g @openai/codex` แล้วกด Enter เพื่อติดตั้ง Codex CLI

3

รอจนติดตั้งเสร็จ แล้วพิมพ์ `codex --version` เพื่อตรวจสอบเวอร์ชันที่ติดตั้ง

4

พิมพ์คำสั่ง `codex` เพื่อเริ่มใช้งานครั้งแรก ระบบจะให้เข้าสู่ระบบ (Login)

5

เลือกเข้าสู่ระบบด้วยบัญชี ChatGPT แล้วทำตามขั้นตอนในเบราว์เซอร์ที่เปิดขึ้นมา

6

เมื่อเข้าสู่ระบบสำเร็จ กลับมาที่ Command Prompt จะพร้อมใช้งาน Codex ได้ทันที



Mac

เปิด Terminal แล้วใช้คำสั่งชุดเดียวกันได้ทุกคำสั่ง (`npm install -g @openai/codex`, `codex --version`, `codex`) เพราะ Codex CLI รองรับ Mac ในรูปแบบเดียวกับ Windows



เคล็ดลับ

ถ้าเจอ error เกี่ยวกับสิทธิ์การติดตั้ง (permission) ให้เปิด Command Prompt แบบ Run as Administrator แล้วลองใหม่อีกครั้ง

Demo การใช้งาน Codex ผ่าน CMD

40 นาที

AI Agent Setup 2 / 2

แนวคิด

- พิมพ์คำสั่ง codex ใน Command Prompt เพื่อเริ่ม Session สันทนากับ AI
- สั่งงานด้วยประโยคภาษาธรรมชาติได้ทันที โดยไม่ต้องเขียนโค้ดเอง
- Codex จะวางแผนและเสนอสิ่งที่ จะทำ ก่อนลงมือแก้ไขไฟล์จริงทุกครั้ง
- เหมาะกับงานข้อมูล เช่น สรุปไฟล์ จัดระเบียบข้อมูล หรือสร้างสคริปต์ช่วยงาน



Command Prompt

```
C:\Users\student> codex
```

```
Codex> สรุปเนื้อหาไฟล์ report.txt ให้หน่อย
```

```
Codex> สร้างไฟล์ summary.csv  
จากข้อมูลใน data.xlsx
```

```
Codex> เขียนสคริปต์เช็คไฟล์ซ้ำ  
ในโฟลเดอร์นี้ให้หน่อย
```

⚠️ ข้อควรระวัง

ตรวจสอบผลลัพธ์ที่ Codex สร้างทุกครั้งก่อนนำไปใช้จริง — AI อาจให้ข้อมูลผิดพลาดได้

10:30 - 12:10

02

Programming for Fun (Node.js)

ลงมือเขียนโปรแกรมจริงตั้งแต่ติดตั้งเครื่องมือจนได้โปรแกรมของตัวเอง

IDE Installation (VS Code) + Extension

10 นาที

ขั้นตอนที่ 2 / 8

1

เข้าเว็บไซต์ code.visualstudio.com

2

คลิกปุ่มดาวน์โหลดสีน้ำเงินตรงกลางหน้าจอ (ระบบจะเลือกเวอร์ชันให้ตรงกับเครื่องอัตโนมัติ)

3

เปิดไฟล์ที่ดาวน์โหลดมา แล้วกด Next / I Agree ไปเรื่อยๆ จนถึงปุ่ม Install

4

เปิดโปรแกรม VS Code ขึ้นมาหลังติดตั้งเสร็จ

5

คลิกไอคอน Extensions ทางแถบซ้าย (รูปสี่เหลี่ยมต่อกัน 4 ชั้น)

6

พิมพ์ค้นหา "JavaScript (ES6) code snippets" แล้วกด Install

7

ค้นหาและติดตั้งเพิ่มอีกตัว: "Code Runner" (ใช้กดรันโค้ดได้ทันทีโดยไม่ต้องพิมพ์คำสั่ง)



Mac

ขั้นตอนเหมือนกันทุกอย่าง เพียงดาวน์โหลดไฟล์ .zip แทน .exe แล้วลาก VS Code ไปไว้ในโฟลเดอร์ Applications



Scan QR Code สำหรับเข้า Link Download



APIDog Installation

20 นาที

ขั้นตอนที่ 3 / 8

- 1 เข้าเว็บไซต์ apidog.com แล้วคลิกปุ่ม Download
- 2 เลือกเวอร์ชันสำหรับ Windows (ระบบจะดาวน์โหลดไฟล์ .exe)
- 3 เปิดไฟล์ติดตั้ง แล้วกด Next / Install ตามค่าเริ่มต้น
- 4 เปิดโปรแกรม APIDog แล้วสมัครสมาชิกฟรี หรือ Skip เพื่อใช้งานแบบไม่ล็อกอิน
- 5 ลองสร้าง Project ใหม่ (New Project) เพื่อทำความเข้าใจกับหน้าจอ
- 6 APIDog คือเครื่องมือช่วยทดสอบ API — เราจะใช้มันดูผลลัพธ์การรันโค้ดของเราในขั้นตอนถัดไป



Mac

ดาวน์โหลดไฟล์ .dmg จากเว็บเดียวกัน ดับเบิลคลิก แล้วลากไอคอน APIDog ไปไว้ในโฟลเดอร์ Applications เหมือนโปรแกรม Mac ทั่วไป

💡 Scan QR Code สำหรับเข้า Link Download



Variable (ตัวแปร)

10 นาที

ขั้นตอนที่ 4 / 8

แนวคิด

- ตัวแปร คือ "กล่องเก็บข้อมูล" ที่มีชื่อ ให้เราเรียกใช้ค่านั้นซ้ำได้
- ใน Node.js ใช้คำว่า let หรือ const นำหน้าชื่อตัวแปร
- let ใช้กับค่าที่เปลี่ยนแปลงได้ / const ใช้กับค่าที่คงที่ไม่เปลี่ยนแปลง
- ข้อมูลมีหลายชนิด เช่น ตัวเลข (Number) และข้อความ (String)



index.js

```
let name = "Nampueng";  
let age = 12;  
const school = "บ้านสวน";  
  
console.log(name);  
console.log(age);  
console.log(school);
```

💡 เคล็ดลับ

ลองเปลี่ยนชื่อในตัวแปร name เป็นชื่อของตัวเอง แล้วรันดูผลลัพธ์!

วิธีรัน: บันทึกไฟล์ แล้วพิมพ์ `node index.js` ใน Terminal ของ VS Code (หรือกด ▶ ปุ่ม Code Runner)

Comparison Operations

10 นาที

ขั้นตอนที่ 5 / 8

แนวคิด

- ตัวดำเนินการเปรียบเทียบ ใช้เพื่อดูว่าค่าสองค่า "จริง (true)" หรือ "เท็จ (false)"
- `==` เท่ากับ `!=` ไม่เท่ากับ
- `>` มากกว่า `<` น้อยกว่า
- `>=` มากกว่าหรือเท่ากับ `<=` น้อยกว่าหรือเท่ากับ
- ผลลัพธ์ที่ได้จะเป็น true หรือ false เสมอ



index.js

```
let score = 75;

console.log(score > 50);
// true
console.log(score == 100);
// false
console.log(score >= 75);
// true
```

💡 เคล็ดลับ

ลองเปลี่ยนค่า score เป็นเลขอื่น แล้วเดาก่อนว่าผลลัพธ์จะเป็น true หรือ false

วิธีรัน: บันทึกไฟล์ แล้วพิมพ์ `node index.js` ใน Terminal ของ VS Code (หรือกด ▶ ปุ่ม Code Runner)

IF ELSE (เงื่อนไข)

10 นาที

ขั้นตอนที่ 6 / 8

แนวคิด

- if...else ใช้ตัดสินใจว่าจะให้โปรแกรมทำอะไร ขึ้นอยู่กับเงื่อนไข
- ถ้าเงื่อนไขใน if เป็นจริง → ทำงานใน { } ของ if
- ถ้าเป็นเท็จ → ข้ามไปทำงานใน { } ของ else แทน
- สามารถใช้ else if เพิ่มได้หลายเงื่อนไขต่อกัน



index.js

```
let score = 75;

if (score >= 80) {
  console.log("เกรด A");
} else if (score >= 60) {
  console.log("เกรด B");
} else {
  console.log("ต้องพยายามอีกนิด");
}
```

💡 เคล็ดลับ

ลองปรับค่า score ให้ผ่านเงื่อนไขแต่ละแบบ เพื่อดูว่าเกรดที่ได้เปลี่ยนไปอย่างไร

วิธีรัน: บันทึกไฟล์ แล้วพิมพ์ `node index.js` ใน Terminal ของ VS Code (หรือกด ▶ ปุ่ม Code Runner)

Loop (For loop)

10 นาที

ขั้นตอนที่ 7 / 8

แนวคิด

- for loop ใช้ทำงานซ้ำๆ โดยไม่ต้องเขียนโค้ดซ้ำหลายบรรทัด
- รูปแบบ: for (เริ่มต้น; เงื่อนไข; เพิ่มค่า) { งานที่ทำซ้ำ }
- i เป็นตัวแปรนับรอบ เริ่มจาก 0 แล้วเพิ่มทีละ 1 (i++)
- loop จะทำงานซ้ำจนกว่าเงื่อนไขจะเป็นเท็จ แล้วจึงหยุด



index.js

```
for (let i = 1; i <= 5; i++) {  
  console.log("รอบที่ " + i);  
}
```

// ผลลัพธ์:

// รอบที่ 1

// รอบที่ 2 ... รอบที่ 5

💡 เคล็ดลับ

ลองเปลี่ยนเลข 5 เป็นเลขอื่น แล้วสังเกตว่าโปรแกรมพิมพ์กี่รอบ

วิธีรัน: บันทึกไฟล์ แล้วพิมพ์ `node index.js` ใน Terminal ของ VS Code (หรือกด ▶ ปุ่ม Code Runner)

แบบทดสอบ: เขียนโปรแกรมคำนวณเกรด

30 นาที

ขั้นตอนที่ 8 / 8 – แบบทดสอบสรุปท้ายบท: รวม Variable + Comparison + IF ELSE + Loop

📄 คำสั่ง

ให้นักศึกษาเขียนโปรแกรมด้วย Node.js ตามโจทย์ต่อไปนี้ด้วยตนเอง

โจทย์

- 1 กำหนดตัวแปร scores เก็บคะแนนสอบของนักศึกษา 5 คน (ตัวเลขเต็ม 0-100)
- 2 ใช้คำสั่งวนซ้ำ (Loop) เพื่ออ่านคะแนนทีละคน
- 3 ใช้เงื่อนไข (if...else) ตัดสินเกรดตามเกณฑ์ที่กำหนด
- 4 แสดงผลลัพธ์ทางหน้าจอในรูปแบบ "คะแนน -> เกรด" ให้ครบทุกคน

เกณฑ์การตัดเกรด

คะแนน >= 80 → เกรด A
60 <= คะแนน < 80 → เกรด B
คะแนน < 60 → ต้องพัฒนาเพิ่มเติม

ตัวอย่างผลลัพธ์ที่คาดหวัง (Input/Output)

Input: [95, 82, 67, 55, 40]

Output:
95 -> เกรด A
82 -> เกรด A
67 -> เกรด B
...

ส่งงาน: บันทึกไฟล์ชื่อ grade.js แล้วส่งผ่านช่องทางที่กำหนด

13:30 – 18:00

03

Workshop Catch me if you can

Let combined all we know

ขั้นตอนที่ 1 / 3 — แบ่งทีมและมอบหมายหน้าที่ก่อนเริ่มเกม

 จัดกลุ่ม — แบ่งผู้เข้าร่วมออกเป็นกลุ่ม กลุ่มละ X คน แล้วกระจายบทบาทตามด้านล่างนี้ภายในแต่ละกลุ่ม

บทบาทในกลุ่ม

 ลงที่ (1 คน)  ปรับตามขนาดกลุ่ม (X คน)

หัวหน้ากลุ่ม	1 คน	นักทดสอบระบบ	X คน
รองหัวหน้ากลุ่ม	1 คน	นัก Present	1 คน
นักเขียนโปรแกรม	X คน	นักประสานงาน	1 คน
นักออกแบบดีไซน์	X คน	ฝ่ายบุคคล	1 คน
นักทำเอกสาร	X คน	ฝ่ายบัญชี	X คน

ขั้นตอนที่ 2 / 3 — รายละเอียดงานที่ทุกกลุ่มต้องส่งมอบเมื่อจบเกม



Project Proposal

เอกสารเสนอโครงการ



Program

ตัวโปรแกรมที่พัฒนาจริง



Prototype System

ระบบต้นแบบที่ใช้งานได้



Presentation

การนำเสนอผลงานหน้าห้อง



Cost and Budget

สรุปต้นทุนและงบประมาณ



Team Member

รายชื่อสมาชิกในทีม

ขั้นตอนที่ 3 / 3 (1 / 2) — สิทธิการทำงานของแต่ละตำแหน่งในเกม

เกรด	Position	ค่าตัว	พบลูกค้า	เสนอราคา	ซื้อขายสมาชิก	ติดต่อ ช.กลาง	นำเสนอ ผลงาน	ติดต่อ Consult	ติดต่อ HR กลาง	ขอย้าย กลุ่ม
SSS	หัวหน้ากลุ่ม	3500	Yes	Yes	No	No	No	No	No	Yes
SS	รองหัวหน้ากลุ่ม	3000	Yes	Yes	No	No	No	No	No	Yes
B	นักเขียนโปรแกรม	1000	No	No	No	No	No	No	No	Yes
C	นักออกแบบดีไซน์	500	No	No	No	No	No	No	No	Yes
A	นักทำเอกสาร	1500	No	No	No	No	No	No	No	Yes
C	นักทดสอบระบบ	500	No	No	No	No	No	No	No	Yes
A	นัก Present	1500	Yes	Yes	No	No	Yes	No	No	Yes
B	นักประสานงาน	1000	Yes	No	No	No	No	Yes	No	Yes
B	ฝ่ายบุคคล	1000	No	No	Yes	No	No	No	Yes	Yes
B	ฝ่ายบัญชี	1000	No	Yes	No	Yes	No	No	No	Yes

เกรดตามค่าตัว: SSS (≥3500) • SS (3000) • A (1500) • B (1000) • C (500)

ขั้นตอนที่ 3 / 3 (2 / 2) — เงื่อนไขการขโมยและคืนเงินระหว่างเกม



ขโมย 1 ครั้ง ใช้เวลา 20 นาที

ก่อนธนาคารโอนเงินให้กลุ่มที่ขโมย



ขโมยได้ครั้งละ 5,000 บาท

คิดดอกเบี้ย 2.5% ต่อการขโมย 1 ครั้ง



คืนเงินภายใน 25 นาที

หลังได้รับเงิน ต้องคืนเงินต้น + ดอกเบี้ย

ตัวอย่าง (Example)

กลุ่ม	จำนวนเงิน ขโมย	เวลาขโมย	ระยะเวลา จ่าย	ธนาคารจ่าย เวลา	ระยะเวลา เรียกคืน	ธนาคารเรียก คืนเวลา	จำนวนเงิน คืน
X	5,000	14:00:00	20 นาที	14:20:00	25 นาที	14:45:00	5,125

* 5,000 บาท + ดอกเบี้ย 2.5% (125 บาท) = คืนเงินทั้งหมด 5,125 บาท